
Computer System Architecture

Lab 5

MIPS Conditional Statements

MIPS Conditional Statements

- We can perform Conditional statements in MIPS assembly using Branching and labels.

MIPS Conditional Statements

C code

```
int g = 5;
int h = 6;
int i = 10;
int j = 11;
int f;
if( i == j ) {
    f = g + h;
}
else{
    f = g - h;
}
```

MIPS Assembly

```
addi $s1 , $0 , 5
addi $s2 , $0 , 6
addi $s3 , $0 , 10
addi $s4 , $0 , 11
bne $s3 , $s4 , Else
add $s0 , $s1 , $s2
j Exit
Else:
sub $s0 , $s1 , $s2
Exit:
```

MIPS Conditional Statements

MIPS Assembly

```
addi $s1 , $0 , 5
addi $s2 , $0 , 6
addi $s3 , $0 , 10
addi $s4 , $0 , 11
bne $s3 , $s4 , Else
add $s0 , $s1 , $s2
j Exit
Else:
sub $s0 , $s1 , $s2
Exit:
```

Registers



MIPS Conditional Statements

MIPS Assembly

```

→ addi $s1 , $0 , 5
  addi $s2 , $0 , 6
  addi $s3 , $0 , 10
  addi $s4 , $0 , 11
  bne $s3 , $s4 , Else
  add $s0 , $s1 , $s2
  j Exit
Else:
  sub $s0 , $s1 , $s2
Exit:
  
```

Registers



MIPS Conditional Statements

MIPS Assembly

```

→ addi $s1 , $0 , 5
  addi $s2 , $0 , 6
  addi $s3 , $0 , 10
  addi $s4 , $0 , 11
  bne $s3 , $s4 , Else
  add $s0 , $s1 , $s2
  j Exit
Else:
  sub $s0 , $s1 , $s2
Exit:
  
```

Registers



MIPS Conditional Statements

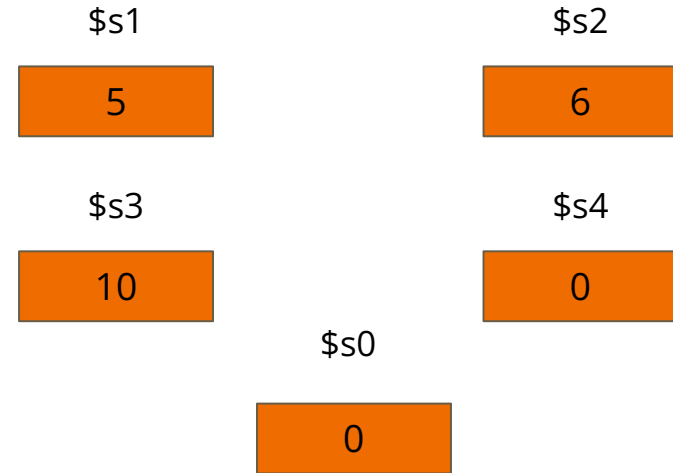
MIPS Assembly

```

addi $s1 , $0 , 5
addi $s2 , $0 , 6
→ addi $s3 , $0 , 10
addi $s4 , $0 , 11
bne $s3 , $s4 , Else
add $s0 , $s1 , $s2
j Exit
Else:
sub $s0 , $s1 , $s2
Exit:

```

Registers



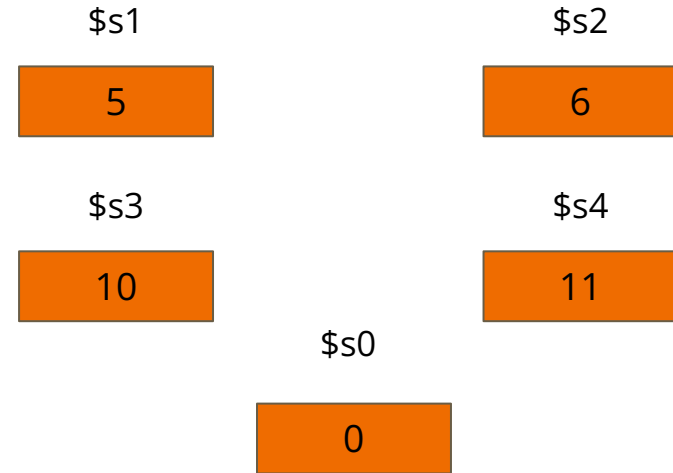
MIPS Conditional Statements

MIPS Assembly

```

addi $s1 , $0 , 5
addi $s2 , $0 , 6
addi $s3 , $0 , 10
→ addi $s4 , $0 , 11
bne $s3 , $s4 , Else
add $s0 , $s1 , $s2
j Exit
Else:
sub $s0 , $s1 , $s2
Exit:
  
```

Registers



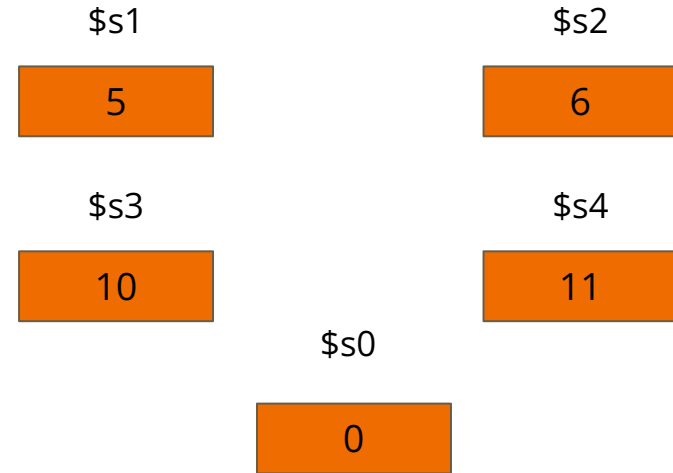
MIPS Conditional Statements

MIPS Assembly

```

addi $s1 , $0 , 5
addi $s2 , $0 , 6
addi $s3 , $0 , 10
addi $s4 , $0 , 11
→ bne $s3 , $s4 , Else
add $s0 , $s1 , $s2
j Exit
Else:
sub $s0 , $s1 , $s2
Exit:
  
```

Registers



bne(Branch if not equal): is an instruction that compares two register values, and performs a branch to the target address if they are not equal. The same functionality can be performed by similar branch instructions like (beq).

MIPS Conditional Statements

MIPS Assembly

```

addi $s1 , $0 , 5
addi $s2 , $0 , 6
addi $s3 , $0 , 10
addi $s4 , $0 , 11
bne $s3 , $s4 , Else
add $s0 , $s1 , $s2
j Exit
Else:
sub $s0 , $s1 , $s2
Exit:
  
```

Registers

\$s1	\$s2
5	6
\$s3	\$s4
10	11
\$s0	
-1	

bne(Branch if not equal): is an instruction that compares two register values, and performs a branch to the target address if they are not equal. The same functionality can be performed by similar branch instructions like (beq).

MIPS Loops

MIPS Loops

- We can also write loops in MIPS assembly using Branching and labels.

MIPS Loops

C code

```
int g = 5;
int h = 10;
int f = 0;
h= h*h;
for(int i = 0 ; i < 3 ; i ++) {
    f += g + h;
}
```

MIPS Assembly

```
addi $t2 , $0 , 5
addi $t3 , $0 , 10
mul $t3 , $t3 , $t3
add $t0 , $zero , $zero
Loop : slti $t1 , $t0 , 3
beq $t1 , $zero , Exit
add $t4 , $t2 , $t3
add $s0 , $t4 , $s0
addi $t0 , $t0 , 1
j Loop
Exit :
```

MIPS Loops

MIPS Assembly

```

addi $t2 , $0 , 5
addi $t3 , $0 , 10
mul  $t3 , $t3 , $t3
add  $t0 , $zero , $zero
Loop : slti $t1 , $t0 , 3
      beq $t1 , $zero , Exit
      add $t4 , $t2 , $t3
      add $s0 , $t4 , $s0
      addi $t0 , $t0 , 1
      j  Loop
Exit :
  
```

Registers

\$t0	\$t1
0	0
\$t2	\$t3
0	0
\$t4	\$s0
0	0

MIPS Loops

MIPS Assembly

```

→ addi $t2 , $0 , 5
  addi $t3 , $0 , 10
  mul $t3 , $t3 , $t3
  add $t0 , $zero , $zero
  Loop : slti $t1 , $t0 , 3
  beq $t1 , $zero , Exit
  add $t4 , $t2 , $t3
  add $s0 , $t4 , $s0
  addi $t0 , $t0 , 1
  j Loop
  Exit :
  
```

Registers

\$t0	\$t1
0	0
\$t2	\$t3
5	0
\$t4	\$s0
0	0

MIPS Loops

MIPS Assembly

```

→ addi $t2 , $0 , 5
  addi $t3 , $0 , 10
  mul  $t3 , $t3 , $t3
  add  $t0 , $zero , $zero
Loop : slti $t1 , $t0 , 3
      beq  $t1 , $zero , Exit
      add  $t4 , $t2 , $t3
      add  $s0 , $t4 , $s0
      addi $t0 , $t0 , 1
      j   Loop
Exit :
  
```

Registers

\$t0	\$t1
0	0
\$t2	\$t3
5	10
\$t4	\$s0
0	0

MIPS Loops

MIPS Assembly

```

addi $t2 , $0 , 5
addi $t3 , $0 , 10
→ mul $t3 , $t3 , $t3
add $t0 , $zero , $zero
Loop : slti $t1 , $t0 , 3
      beq $t1 , $zero , Exit
      add $t4 , $t2 , $t3
      add $s0 , $t4 , $s0
      addi $t0 , $t0 , 1
      j Loop
Exit :
  
```

Registers

\$t0	\$t1
0	0
\$t2	\$t3
5	100
\$t4	\$s0
0	0

MIPS Loops

MIPS Assembly

```

addi $t2 , $0 , 5
addi $t3 , $0 , 10
mul $t3 , $t3 , $t3
→ add $t0 , $zero , $zero
Loop : slti $t1 , $t0 , 3
      beq $t1 , $zero , Exit
      add $t4 , $t2 , $t3
      add $s0 , $t4 , $s0
      addi $t0 , $t0 , 1
      j Loop
Exit :
  
```

Registers

\$t0	\$t1
0	0
\$t2	\$t3
5	100
\$t4	\$s0
0	0

Initialize i(counter)

MIPS Loops

MIPS Assembly

```

addi $t2 , $0 , 5
addi $t3 , $0 , 10
mul  $t3 , $t3 , $t3
add  $t0 , $zero , $zero
→ Loop : slti $t1 , $t0 , 3
      beq $t1 , $zero , Exit
      add $t4 , $t2 , $t3
      add $s0 , $t4 , $s0
      addi $t0 , $t0 , 1
      j  Loop
Exit :
  
```

Registers

\$t0	\$t1
0	1
\$t2	\$t3
5	100
\$t4	\$s0
0	0

slti(set less than immediate): is an instruction that sets the 1st argument(register) to 1 if the 2nd argument(register) is less than the 3rd argument (immediate value), else the 1st argument register is set to 0.
 We use this instruction to check if i is less than the 3 in this case.

MIPS Loops

MIPS Assembly

```

addi $t2 , $0 , 5
addi $t3 , $0 , 10
mul $t3 , $t3 , $t3
add $t0 , $zero , $zero
Loop : slti $t1 , $t0 , 3
→ beq $t1 , $zero , Exit
add $t4 , $t2 , $t3
add $s0 , $t4 , $s0
addi $t0 , $t0 , 1
j Loop
Exit :

```

Registers

\$t0	\$t1
0	1
\$t2	\$t3
5	100
\$t4	\$s0
0	0

We branch using beq (branches if the the 2 register values are equal) to exit the loop if \$t0 (the counter) is equal to or larger than 3. In this iteration we dont branch

MIPS Loops

MIPS Assembly

```

addi $t2 , $0 , 5
addi $t3 , $0 , 10
mul $t3 , $t3 , $t3
add $t0 , $zero , $zero
Loop : slti $t1 , $t0 , 3
      beq $t1 , $zero , Exit
      → add $t4 , $t2 , $t3
        add $s0 , $t4 , $s0
        addi $t0 , $t0 , 1
        j Loop
Exit :

```

Registers

\$t0	\$t1
0	1
\$t2	\$t3
5	100
\$t4	\$s0
105	0

MIPS Loops

MIPS Assembly

```

addi $t2 , $0 , 5
addi $t3 , $0 , 10
mul $t3 , $t3 , $t3
add $t0 , $zero , $zero
Loop : slti $t1 , $t0 , 3
      beq $t1 , $zero , Exit
      add $t4 , $t2 , $t3
      → add $s0 , $t4 , $s0
      addi $t0 , $t0 , 1
      j Loop
Exit :
  
```

Registers

\$t0	\$t1
0	1
\$t2	\$t3
5	100
\$t4	\$s0
105	105

MIPS Loops

MIPS Assembly

```

addi $t2 , $0 , 5
addi $t3 , $0 , 10
mul $t3 , $t3 , $t3
add $t0 , $zero , $zero
Loop : slti $t1 , $t0 , 3
      beq $t1 , $zero , Exit
      add $t4 , $t2 , $t3
      add $s0 , $t4 , $s0
      addi $t0 , $t0 , 1
      j Loop
Exit :
  
```

Registers

\$t0	\$t1
1	1
\$t2	\$t3
5	100
\$t4	\$s0
105	105

Increment counter

MIPS Loops

MIPS Assembly

```

addi $t2 , $0 , 5
addi $t3 , $0 , 10
mul  $t3 , $t3 , $t3
add  $t0 , $zero , $zero
Loop : slti $t1 , $t0 , 3
      beq $t1 , $zero , Exit
      add $t4 , $t2 , $t3
      add $s0 , $t4 , $s0
      addi $t0 , $t0 , 1
      → j Loop
Exit :
  
```

Registers

\$t0	\$t1
1	1
\$t2	\$t3
5	100
\$t4	\$s0
105	105

Jump back to the beginning of the loop

MIPS Loops

MIPS Assembly

```

addi $t2 , $0 , 5
addi $t3 , $0 , 10
mul $t3 , $t3 , $t3
add $t0 , $zero , $zero
→ Loop : slti $t1 , $t0 , 3
beq $t1 , $zero , Exit
add $t4 , $t2 , $t3
add $s0 , $t4 , $s0
addi $t0 , $t0 , 1
j Loop
Exit :
  
```

Registers

\$t0	\$t1
1	1
\$t2	\$t3
5	100
\$t4	\$s0
105	105

MIPS Loops

MIPS Assembly

```

addi $t2 , $0 , 5
addi $t3 , $0 , 10
mul  $t3 , $t3 , $t3
add  $t0 , $zero , $zero
Loop : slti $t1 , $t0 , 3
→ beq $t1 , $zero , Exit
add  $t4 , $t2 , $t3
add  $s0 , $t4 , $s0
addi $t0 , $t0 , 1
j    Loop
Exit :
  
```

Registers

\$t0	\$t1
1	1
\$t2	\$t3
5	100
\$t4	\$s0
105	105

We branch to exit the loop if \$t0(the counter) is equal to or larger than 3. In this iteration we dont branch

MIPS Loops

MIPS Assembly

```

addi $t2 , $0 , 5
addi $t3 , $0 , 10
mul  $t3 , $t3 , $t3
add  $t0 , $zero , $zero
Loop : slti $t1 , $t0 , 3
      beq $t1 , $zero , Exit
      → add $t4 , $t2 , $t3
      add $s0 , $t4 , $s0
      addi $t0 , $t0 , 1
      j  Loop
Exit :
  
```

Registers

\$t0	\$t1
1	1
\$t2	\$t3
5	100
\$t4	\$s0
105	105

MIPS Loops

MIPS Assembly

```

addi $t2 , $0 , 5
addi $t3 , $0 , 10
mul  $t3 , $t3 , $t3
add  $t0 , $zero , $zero
Loop : slti $t1 , $t0 , 3
      beq $t1 , $zero , Exit
      add $t4 , $t2 , $t3
      → add $s0 , $t4 , $s0
      addi $t0 , $t0 , 1
      j Loop
Exit :
  
```

Registers

\$t0	\$t1
1	1
\$t2	\$t3
5	100
\$t4	\$s0
105	210

MIPS Loops

MIPS Assembly

```

addi $t2 , $0 , 5
addi $t3 , $0 , 10
mul  $t3 , $t3 , $t3
add  $t0 , $zero , $zero
Loop : slti $t1 , $t0 , 3
      beq $t1 , $zero , Exit
      add $t4 , $t2 , $t3
      add $s0 , $t4 , $s0
      addi $t0 , $t0 , 1
      j Loop
Exit :

```

Registers

\$t0	\$t1
2	1
\$t2	\$t3
5	100
\$t4	\$s0
105	210

Increment Counter

MIPS Loops

MIPS Assembly

```

addi $t2 , $0 , 5
addi $t3 , $0 , 10
mul $t3 , $t3 , $t3
add $t0 , $zero , $zero
→ Loop : slti $t1 , $t0 , 3
beq $t1 , $zero , Exit
add $t4 , $t2 , $t3
add $s0 , $t4 , $s0
addi $t0 , $t0 , 1
j Loop
Exit :
  
```

Registers

\$t0	\$t1
2	1
\$t2	\$t3
5	100
\$t4	\$s0
105	105

MIPS Loops

MIPS Assembly

```

addi $t2 , $0 , 5
addi $t3 , $0 , 10
mul  $t3 , $t3 , $t3
add  $t0 , $zero , $zero
Loop : slti $t1 , $t0 , 3
      → beq $t1 , $zero , Exit
      add $t4 , $t2 , $t3
      add $s0 , $t4 , $s0
      addi $t0 , $t0 , 1
      j Loop
Exit :
  
```

Registers

\$t0	\$t1
2	1
\$t2	\$t3
5	100
\$t4	\$s0
105	105

We branch to exit the loop if \$t0(the counter) is equal to or larger than 3. In this iteration we dont branch

MIPS Loops

MIPS Assembly

```

addi $t2 , $0 , 5
addi $t3 , $0 , 10
mul  $t3 , $t3 , $t3
add  $t0 , $zero , $zero
Loop : slti $t1 , $t0 , 3
      beq $t1 , $zero , Exit
      → add $t4 , $t2 , $t3
      add $s0 , $t4 , $s0
      addi $t0 , $t0 , 1
      j  Loop
Exit :
  
```

Registers

\$t0	\$t1
2	1
\$t2	\$t3
5	100
\$t4	\$s0
105	105

MIPS Loops

MIPS Assembly

```

addi $t2 , $0 , 5
addi $t3 , $0 , 10
mul  $t3 , $t3 , $t3
add  $t0 , $zero , $zero
Loop : slti $t1 , $t0 , 3
      beq $t1 , $zero , Exit
      add $t4 , $t2 , $t3
      → add $s0 , $t4 , $s0
      addi $t0 , $t0 , 1
      j Loop
Exit :
  
```

Registers

\$t0	\$t1
2	1
\$t2	\$t3
5	100
\$t4	\$s0
105	315

MIPS Loops

MIPS Assembly

```

addi $t2 , $0 , 5
addi $t3 , $0 , 10
mul  $t3 , $t3 , $t3
add  $t0 , $zero , $zero
Loop : slti $t1 , $t0 , 3
      beq $t1 , $zero , Exit
      add $t4 , $t2 , $t3
      add $s0 , $t4 , $s0
      ← addi $t0 , $t0 , 1
      j  Loop
Exit :
  
```

Registers

\$t0	\$t1
3	1
\$t2	\$t3
5	100
\$t4	\$s0
105	315

Increment Counter

MIPS Loops

MIPS Assembly

```

addi $t2 , $0 , 5
addi $t3 , $0 , 10
mul  $t3 , $t3 , $t3
add  $t0 , $zero , $zero
Loop : slti $t1 , $t0 , 3
      beq $t1 , $zero , Exit
      add $t4 , $t2 , $t3
      add $s0 , $t4 , $s0
      addi $t0 , $t0 , 1
      → j Loop
Exit :
  
```

Registers

\$t0	\$t1
3	1
\$t2	\$t3
5	100
\$t4	\$s0
105	315

Jump back to the beginning of the loop

MIPS Loops

MIPS Assembly

```

addi $t2 , $0 , 5
addi $t3 , $0 , 10
mul  $t3 , $t3 , $t3
add  $t0 , $zero , $zero
→ Loop : slti $t1 , $t0 , 3
      beq $t1 , $zero , Exit
      add $t4 , $t2 , $t3
      add $s0 , $t4 , $s0
      addi $t0 , $t0 , 1
      j  Loop
Exit :
  
```

Registers

\$t0	\$t1
3	0
\$t2	\$t3
5	100
\$t4	\$s0
105	315

The counter is no longer less than 3, \$t1 is set to 0

MIPS Loops

MIPS Assembly

```

addi $t2 , $0 , 5
addi $t3 , $0 , 10
mul  $t3 , $t3 , $t3
add  $t0 , $zero , $zero
Loop : slti $t1 , $t0 , 3
      → beq $t1 , $zero , Exit
      add $t4 , $t2 , $t3
      add $s0 , $t4 , $s0
      addi $t0 , $t0 , 1
      j Loop
Exit :
  
```

Registers

\$t0	\$t1
3	0
\$t2	\$t3
5	100
\$t4	\$s0
105	315

We branch to exit the loop if \$t0(the counter) is equal to or larger than 3. In this we branch to exit terminating the loop

MIPS Loops

MIPS Assembly

```

addi $t2 , $0 , 5
addi $t3 , $0 , 10
mul  $t3 , $t3 , $t3
add  $t0 , $zero , $zero
Loop : slti $t1 , $t0 , 3
      beq $t1 , $zero , Exit
      add $t4 , $t2 , $t3
      add $s0 , $t4 , $s0
      addi $t0 , $t0 , 1
      j Loop
      Exit :
  
```

Registers

\$t0	\$t1
3	0
\$t2	\$t3
5	100
\$t4	\$s0
105	315

We branch to exit the loop if \$t0(the counter) is equal to or larger than 3. In this we branch to exit terminating the loop

MIPS functions

- C and JAVA programs that make calls to a function can also be converted to and implemented in MIPS Instructions for the CPU to understand and execute them

main:

```
addi $a0,$zero,10
```

```
addi $a1,$zero,20
```

```
jal addnum
```

```
mul $s0,$v1,2
```

```
addi $v0,$zero,10
```

```
syscall
```

addnum:

```
add $v1,$a0,$a1
```

```
jr $ra
```

High level functions

```
int sum (int a, int b)
```

Arguments of the function

```
{
```

```
    return a + b;
```

Return value of the function

```
}
```

High level functions

```
int sum (int a, int b)
```

Stored in \$a registers in MIPS processors

```
{
```

```
    return a + b;
```

Stored in \$v registers in MIPS processors

```
}
```

MIPS functions

Do not forget !: instructions are sequential, they execute in order. So we will have to use an instruction that will make us “jump” from the main program to the function and then “jump back” to the main program”

main:

```
addi $a0,$zero,10
```

```
addi $a1,$zero,20
```

```
jal addnum
```

```
mul $s0,$v1,2
```

```
addi $v0,$zero,10
```

```
syscall
```

addnum:

```
add $v1,$a0,$a1
```

```
jr $ra
```

Instructions in memory

main:

`addi $a0,$zero,10` →

`addi $a1,$zero,20`

`jal addnum`

`mul $s0,$v1,2`

`addi $v0,$zero,10`

`syscall`

addnum:

`add $v1,$a0,$a1`

`jr $ra` →

`addi $a0,$zero,10`

`addi $a1,$zero,20`

`jal addnum`

`mul $s0,$v1,2`

`addi $v0,$zero,10`

`syscall`

`add $v1,$a0,$a1`

`jr $ra`

Instructions in memory

Memory

addi \$a0,\$zero,10	0x01
addi \$a1,\$zero,20	0x02
jal addnum	0x03
mul \$s0,\$v1,2	0x04
addi \$v0,\$zero,10	0x05
syscall	0x06
add \$v1,\$a0,\$a1	0x07 (addnum)
jr \$ra	0x08

Registers

\$a0	\$a1
0	0
\$v0	\$v1
0	0
\$ra	\$pc
	0x01

Instructions in memory

Memory

→	addi \$a0,\$zero,10	0x01
	addi \$a1,\$zero,20	0x02
	jal addnum	0x03
	mul \$s0,\$v1,2	0x04
	addi \$v0,\$zero,10	0x05
	syscall	0x06
	add \$v1,\$a0,\$a1	0x07 (addnum)
	jr \$ra	0x08

Registers

\$a0	\$a1
10	0
\$v0	\$v1
0	0
\$ra	\$pc
	0x02

Instructions in memory

Memory

addi \$a0,\$zero,10	0x01
addi \$a1,\$zero,20	0x02
→ jal addnum	0x03
mul \$s0,\$v1,2	0x04
addi \$v0,\$zero,10	0x05
syscall	0x06
add \$v1,\$a0,\$a1	0x07 (addnum)
jr \$ra	0x08

Registers



Jal (jump and link instruction) : copies the address of the next instruction in the main program from pc To ra (return address register) and then jumps to the first instruction in the function

Instructions in memory

Memory

addi \$a0,\$zero,10	0x01
addi \$a1,\$zero,20	0x02
→ jal addnum	0x03
mul \$s0,\$v1,2	0x04
addi \$v0,\$zero,10	0x05
syscall	0x06
add \$v1,\$a0,\$a1	0x07 (addnum)
jr \$ra	0x08

Registers

\$a0	\$a1
10	20
\$v0	\$v1
0	0
\$ra	\$pc
0x04	0x07

Jal (jump and link instruction) : copies the address of the next instruction in the main program from pc To ra (return address register) and then jumps to the first instruction in the function

Instructions in memory

Memory

addi \$a0,\$zero,10	0x01
addi \$a1,\$zero,20	0x02
jal addnum	0x03
mul \$s0,\$v1,2	0x04
addi \$v0,\$zero,10	0x05
syscall	0x06
add \$v1,\$a0,\$a1	0x07 (addnum)
jr \$ra	0x08



Registers

\$a0	\$a1
10	20
\$v0	\$v1
0	30
\$ra	\$pc
0x04	0x08

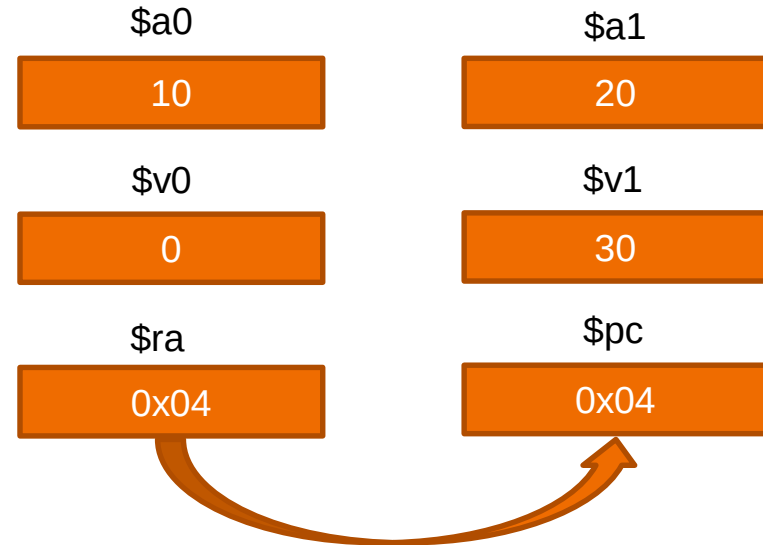
Instructions in memory

Memory

addi \$a0,\$zero,10	0x01
addi \$a1,\$zero,20	0x02
jal addnum	0x03
mul \$s0,\$v1,2	0x04
addi \$v0,\$zero,10	0x05
syscall	0x06
add \$v1,\$a0,\$a1	0x07 (addnum)
jr \$ra	0x08



Registers



Jr (jump register unconditionally) : copies the address in ra register to pc register to make the program Jump back to the main program at the point it stopped at

Instructions in memory

Memory

addi \$a0,\$zero,10	0x01
addi \$a1,\$zero,20	0x02
jal addnum	0x03
mul \$s0,\$v1,2	0x04
addi \$v0,\$zero,10	0x05
syscall	0x06
add \$v1,\$a0,\$a1	0x07 (addnum)
jr \$ra	0x08

Registers

\$a0	\$a1
10	20
\$v0	\$v1
0	30
\$ra	\$pc
0x04	0x05
	\$s0
	60

Instructions in memory

Memory

addi \$a0,\$zero,10	0x01
addi \$a1,\$zero,20	0x02
jal addnum	0x03
mul \$s0,\$v1,2	0x04
addi \$v0,\$zero,10	0x05
syscall	0x06
add \$v1,\$a0,\$a1	0x07 (addnum)
jr \$ra	0x08

Registers

\$a0	\$a1
10	20
\$v0	\$v1
10	30
\$ra	\$pc
0x04	0x06
	\$s0
	60

Instructions in memory

Memory

addi \$a0,\$zero,10	0x01
addi \$a1,\$zero,20	0x02
jal addnum	0x03
mul \$s0,\$v1,2	0x04
addi \$v0,\$zero,10	0x05
syscall	0x06
add \$v1,\$a0,\$a1	0x07 (addnum)
jr \$ra	0x08



Registers

\$a0	\$a1
10	20
\$v0	\$v1
10	30
\$ra	\$pc
0x04	0x07
	\$s0
	60

Syscall is an essential instruction to terminate the program, otherwise the instruction at 0x07 will be Executed again (according to the pc register) . If the value inside v0 is 10, program will be terminated

Thank you!
